

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Санкт-Петербургский национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине
«Операционные системы»

Базовый трек
Вариант: Windows, Clock, io-thpt-write

Выполнил:

Студент группы Р3316
Митичев Иван Дмитриевич

Преподаватель:

Гиниятуллин Арслан

Оглавление

Задание	3
Ограничения.....	3
Требования к отчету и защите	4
Темы для подготовки.....	4
Исходный код	5
Основные структуры данных	5
Основные функции.....	5
Сравнение.....	7
Вывод.....	8

Задание

Для оптимизации работы с блочными устройствами в ОС существует кэш страниц с данными, которыми мы производим операции чтения и записи на диск. Такой кэш позволяет избежать высоких задержек при повторном доступе к данным, так как операция будет выполнена с данными в RAM, а не на диске (вспомним пирамиду памяти).

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки (dll или so). Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

При выполнении работы необходимо реализовать простой API для работы с файлами, предоставляющий пользователю следующие возможности:

1. Открытие файла по заданному пути файла, доступного для чтения. Процедура возвращает некоторый хэндл на файл. Пример:
`int lab2_open(const char *path).`
2. Закрытие файла по хэндлу. Пример:
`int lab2_close(int fd).`
3. Чтение данных из файла. Пример:
`ssize_t lab2_read(int fd, void buf[.count], size_t count).`
4. Запись данных в файл. Пример:
`ssize_t lab2_write(int fd, const void buf[.count], size_t count).`
5. Перестановка позиции указателя на данные файла. Достаточно поддерживать только абсолютные координаты. Пример:
`off_t lab2_lseek(int fd, off_t offset, int whence).`
6. Синхронизация данных из кэша с диском. Пример:
`int lab2_fsync(int fd).`

Операции с диском разработанного блочного кэша должны производиться в обход page cache используемой ОС.

В рамках проверки работоспособности разработанного блочного кэша необходимо адаптировать указанную преподавателем программу-загрузчик из ЛР 1, добавив использование кэша. Запустите программу и убедитесь, что она корректно работает. Сравните производительность до и после.

Ограничения

- Программа (комплекс программ) должна быть реализован на языке C или C++.
- Если по выданному варианту задана политика вытеснения Optimal, то необходимо предоставить пользователю возможность подсказать page cache, когда будет совершен следующий доступ к данным. Это можно сделать либо добавив параметр в процедуры read и write (например, `ssize_t lab2_read(int fd, void buf[.count], size_t count, access_hint_t hint)`), либо добавив еще одну функцию

в API (например, `int lab2_advice(int fd, off_t offset, access_hint_t hint)`). `access_hint_t` в данном случае – это абсолютное время или временной интервал, по которому разработанное API будет определять время последующего доступа к данным.

- Запрещено использовать высокоуровневые абстракции над системными вызовами. Необходимо использовать, в случае Unix, процедуры `libc`.

Требования к отчету и защите

Отчет должен содержать:

титальный лист с указанием номера и названия ЛР, вашего ФИО, ФИО преподавателя практики, номера вашей группы, варианта ЛР;

текст задания в соответствии с вариантом;

краткий обзор кода;

данные о работе программы-нагрузчика до и после внедрения своего page cache;

заключение с анализом результатов и выводом

Темы для подготовки

Файловые системы в Linux, структура файла, взаимодействие ФС и диска;

Файловая система `ext4`;

Page cache в ОС;

Политики вытеснения кэшей;

Динамические библиотеки в Linux;

Исходный код

<https://github.com/secs-dev-os-course/diy-page-cache-Karabas890/tree/lab2>

Библиотека `cache.h` реализует кэширование файловых данных с политикой вытеснения блоков CLOCK, что позволяет уменьшить количество операций чтения/записи с диска.

Основные структуры данных

1. **CachePage** — описывает страницу кеша:

- `offset` — смещение страницы в файле.
- `size` — размер данных.
- `data` — указатель на данные.
- `is_dirty` — флаг, указывающий, что данные изменены и должны быть записаны на диск.
- `is_referenced` — флаг для алгоритма Clock (используется для вытеснения страниц).

2. **Cache** — кеш для файлов:

- `pages[size_of_cache]` — массив страниц.
- `hand` — указатель на текущую страницу (для алгоритма Clock).
- `lock` — критическая секция для синхронизации доступа к кешу.

3. **File** — файл с кешем:

- `handle` — дескриптор файла.
- `position` — текущая позиция в файле.
- `cache` — кеш, связанный с файлом.

4. **opened_files[max_open_files]** — массив открытых файлов.

5. **files_lock** — критическая секция для работы со списком открытых файлов

Основные функции

Работа с кешем

- `initialize_cache(Cache *cache)`
Обнуляет кеш, устанавливает начальный указатель `hand` для алгоритма Clock.
- `destroy_cache(Cache *cache)`
Освобождает выделенную память для всех страниц кеша.
- `find_in_cache(Cache *cache, LARGE_INTEGER offset)`
Ищет страницу с заданным смещением в кеше.
- `get_or_create_page(File *file, LARGE_INTEGER page_offset, size_t size)`
Ищет страницу в кеше. Если нет — выбирает страницу для замены (Clock), записывает её на диск при `is_dirty = 1`, читает новые данные с диска.

- `clock_cache(Cache *cache)`
Реализует алгоритм Clock для вытеснения страниц.
 - Если страница `is_referenced = 0` — выбирается для замены.
 - Если `is_referenced = 1`, обнуляется `is_referenced` и продолжается поиск.
- `flush_page(File *file, CachePage *page)`
Записывает изменённую страницу на диск, сбрасывает флаг `is_dirty`.

Операции с файлами

- `lab2_read(int fd, void *buffer, size_t count)`
Читает `count` байтов в `buffer`, загружая страницы в кеш по необходимости.
- `lab2_write(int fd, const void *buffer, size_t count)`
Записывает данные, обновляя кеш. Устанавливает `is_dirty = 1`.
- `lab2_open(const char *path)`
Открывает или создает файл, выделяет память под структуру `File`, инициализирует кеш, добавляет файл в `opened_files`, возвращает файловый дескриптор (`fd`).
- `lab2_close(int fd)`
Закрывает файл: сбрасывает кеш на диск (`lab2_fsync`), освобождает кеш, закрывает файловый дескриптор, удаляет файл из `opened_files`.
- `lab2_lseek(int fd, int64_t offset, int whence)`
Устанавливает позицию в файле (аналог `lseek`).
- `lab2_fsync(int fd)`
Сбрасывает все изменённые (`is_dirty = 1`) страницы кеша на диск.

Сравнение

Мы можем видеть, что программа-загрузчик по записи бинарных данных в файл с кэшем пусть и не значительно, но быстрее программы без кэша (при количестве блоков для записи в 10000, 100000 и 1000000, что соответствует 160000 Кб, 1600000 Кб и 16000000 Кб).

```
basti@LAPTOP-S8U2ESI2 MINGW64 /c/учеба/OC2/diy-page-cache-Karabas890 (lab2)
$ ./build/app/Debug/io-thpt-writeModApp.exe 10000
Количество блоков: 10000
Размер блока: 16384 байт
Общее время записи с кешем: 0.253953 секунд
Пропускная способность с кешем: 615.272 MB/s

basti@LAPTOP-S8U2ESI2 MINGW64 /c/учеба/OC2/diy-page-cache-Karabas890 (lab2)
$ ./build/app/Debug/io-thpt-writeApp.exe 10000
Количество блоков: 10000
Размер блока: 16384 байт
Общее время записи: 0.392432 секунд
Пропускная способность: 398.158 MB/s
```

```
basti@LAPTOP-S8U2ESI2 MINGW64 /c/учеба/OC2/diy-page-cache-Karabas890 (main)
$ ./build/app/Debug/io-thpt-writeModApp.exe 100000
Количество блоков: 100000
Размер блока: 16384 байт
Общее время записи: 5.60036 секунд
Пропускная способность: 279 MB/s

basti@LAPTOP-S8U2ESI2 MINGW64 /c/учеба/OC2/diy-page-cache-Karabas890 (main)
$ ./build/app/Debug/io-thpt-writeApp.exe 100000
Количество блоков: 100000
Размер блока: 16384 байт
Общее время записи: 6.88652 секунд
Пропускная способность: 226.892 MB/s
```

```
basti@LAPTOP-S8U2ESI2 MINGW64 /c/учеба/OC2/diy-page-cache-Karabas890 (lab2)
$ ./build/app/Debug/io-thpt-writeModApp.exe 1000000
Количество блоков: 1000000
Размер блока: 16384 байт
Общее время записи с кешем: 118.205 секунд
Пропускная способность с кешем: 132.186 MB/s

basti@LAPTOP-S8U2ESI2 MINGW64 /c/учеба/OC2/diy-page-cache-Karabas890 (lab2)
$ ./build/app/Debug/io-thpt-writeApp.exe 1000000
Количество блоков: 1000000
Размер блока: 16384 байт
Общее время записи: 132.415 секунд
Пропускная способность: 118 MB/s
```

Вывод

В ходе выполнения лабораторной работы был реализован блочный кэш в пространстве пользователя в виде динамической библиотеки, обеспечивающий работу с файлами. Разработанное API позволяет выполнять операции открытия, чтения, записи, синхронизации и закрытия файлов с использованием кэша, управляемого на основе заданной политики вытеснения страниц (CLOCK). Реализация гарантирует эффективность ввода-вывода благодаря работе с данными через RAM, минимизируя обращения к диску. Корректность и производительность решения были проверены путем адаптации программы-загрузчика, продемонстрировав улучшение времени выполнения операций при многократном доступе к одним и тем же данным.